

C++ONLINE

AMLAL EL MAHROUSS

OPEN CONTENT:

THE TPROC LIBRARY

TEXT PROCESSING IN C++

2026

DISCLAIMER:

These slides have been amended to fit the recording format for C++Online, I also hope this helps you (the viewer) understand TProc better!

The Design Rationale:

amlal@nekernel.org

Design Rationale:

TProc aims for:

- Modular-first design, using a ``basic_rope`` container.
- Examples-driven by ready-to-use C++ snippets.
- Free functions for STL compatibility.
- And a new pattern caled 'predictors' (predicate + functor) to execute commands on ropes.

Hands on TProc:

amlal@nekernel.org

The following:

Is an example available on TProc's sources. You can try it, build it, and run it.

Please use a modern C++ compiler for that regard, and MSVC hasn't been tested yet as a disclaimer.

What we're about to study:

```
int main()
{
    auto root      = tproc::crope("The Quick Brown Fox Jumps Over The Lazy Dog");
    auto new_elem  = std::make_unique<tproc::crope>(", and Jumps again");
    auto new_elem_2 = std::make_unique<tproc::crope>(", and then Jumps down.");

    boost::asio::io_context ioc{1};
    auto spawn_strand = boost::asio::make_strand(ioc);

    boost::asio::co_spawn(spawn_strand, [&new_elem, &new_elem_2, &rope]() ->
boost::asio::awaitable<void> {
    root.concat(new_elem.get());
    new_elem->concat(new_elem_2.get());

    co_return;
}, boost::asio::detached);

ocl::asio::run<[]>() { (void)0; }>(ioc);

std::cout << root << std::endl;
}
```



```
int main()
{
    auto root      = tproc::crope("The Quick Brown Fox Jumps Over The Lazy Dog");
    auto new_elem  = std::make_unique<tproc::crope>(", and Jumps again");
    auto new_elem_2 = std::make_unique<tproc::crope>(", and then Jumps down.");

    boost::asio::io_context ioc{1};
    auto spawn_strand = boost::asio::make_strand(ioc);

    boost::asio::co_spawn(spawn_strand, [&new_elem, &new_elem_2, &rope]() ->
boost::asio::awaitable<void> {
    root.concat(new_elem.get());
    new_elem->concat(new_elem_2.get());

    co_return;
}, boost::asio::detached);

    ocl::asio::run<[]>() { (void)0; }>(ioc);

    std::cout << root << std::endl;
}
```



```
int main()
{
    auto root      = tproc::crope("The Quick Brown Fox Jumps Over The Lazy Dog");
    auto new_elem  = std::make_unique<tproc::crope>(", and Jumps again");
    auto new_elem_2 = std::make_unique<tproc::crope>(", and then Jumps down.");
```

```
    boost::asio::io_context ioc{1};
    auto spawn_strand = boost::asio::make_strand(ioc);
```

Allocates three ASCII
ropes.

```
    boost::asio::co_spawn(spawn_strand, [&new_elem, &new_elem_2, &root]() ->
```

```
    boost::asio::awaitable<void> {
        root.concat(new_elem.get());
        new_elem->concat(new_elem_2.get());
```

Concatenates root and
new_elem with elem2.

```
        co_return;
    }, boost::asio::detached);
```

```
    ocl::asio::run<[]>() { (void)0; }>(ioc);
    std::cout << root << std::endl;
```

Prints "root" to the standard
output.

```
}
```

Where can I find the sources?

Project's repository: <https://git.ocl.nekernel.org/tproc>

Project's site: <https://nekernel.org>

Questions?

amlal@nekernel.org

Questions?

Reach out to me on LinkedIn! (amlal-el-mahrouss)

Or via email: amlal@nekernel.org

I'll try my best to reply to your inquiries!